Delft University of Technology



SC42155 – Modeling of Dynamical Systems

Mini Project – Modeling a 3D Printer

Elyas Mzoughi (6798934) - Group 38

Academic Year 2026–2027

GitHub Repository

All project files and Python scripts are available in the following GitHub repository:

`https://github.com/emzoughi/MDS-Mini-Project`

Q1.1 – Kinematics

The CoreXY mechanism uses two motors to control the position of the print head. We assume that the belts are inextensible, that there is no slipping between the belts and pulleys, and that both driving pulleys have the same radius r_m.

Let θ_l and θ_r denote the angular positions of the left and right motors respectively. The linear displacement of a belt driven by a pulley is given by $s = r_m\theta$.

Due to the belt arrangement, horizontal motion requires both motors to rotate in the same direction, whereas vertical motion requires them to rotate in opposite directions. Thus, the belt displacement constraints are

$$\begin{cases} r_m\Delta\theta_l = \Delta x + \Delta y, \\ r_m\Delta\theta_r = \Delta x - \Delta y. \end{cases} \tag{1}$$

Adding and subtracting these equations gives

$$\begin{cases} \Delta x = \dfrac{r_m}{2}(\Delta\theta_l + \Delta\theta_r), \\ \Delta y = \dfrac{r_m}{2}(\Delta\theta_l - \Delta\theta_r). \end{cases} \tag{2}$$

Taking the origin of the coordinate system as the reference position corresponding to $\theta_l = \theta_r = 0$, the print head coordinates become

$$\boxed{\begin{aligned} x &= \frac{r_m}{2}(\theta_l + \theta_r), \\ y &= \frac{r_m}{2}(\theta_l - \theta_r). \end{aligned}} \tag{3}$$

Differentiating with respect to time gives the corresponding velocities

$$\boxed{\begin{aligned} \dot{x} &= \frac{r_m}{2}(\dot{\theta}_l + \dot{\theta}_r), \\ \dot{y} &= \frac{r_m}{2}(\dot{\theta}_l - \dot{\theta}_r). \end{aligned}} \tag{4}$$

In matrix form

$$\begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = \frac{r_m}{2} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} \dot{\theta}_l \\ \dot{\theta}_r \end{bmatrix}. \tag{5}$$

Q1.2 – Dynamics

Electrical dynamics For each motor $j \in \{l, r\}$, we chose to neglect the back electromotive force. Kirchhoff's voltage law and the electromechanical torque relation give

$$u_{m,j} = L_m \frac{d}{dt} i_{m,j} + R_m i_{m,j}, \qquad \tau_j = K_m i_{m,j}. \tag{6}$$

Mechanical dynamics Using the relations derived in Q1.1, the total kinetic energy of the two motor inertias and the print head is

$$K = K_{motors} + K_{head} \tag{7}$$
$$= \frac{J_m}{2}(\dot{\theta}_l^2 + \dot{\theta}_r^2) + \frac{m}{2}(\dot{x}^2 + \dot{y}^2) \tag{8}$$
$$= \frac{J_m}{2}(\dot{\theta}_l^2 + \dot{\theta}_r^2) + \frac{m}{2}\frac{r_m^2}{2}(\dot{\theta}_l^2 + \dot{\theta}_r^2) \tag{9}$$
$$= \frac{J_{\text{eff}}}{2}(\dot{\theta}_l^2 + \dot{\theta}_r^2), \qquad J_{\text{eff}} = J_m + \frac{m r_m^2}{2}. \tag{10}$$

Bearing friction We define the Rayleigh dissipation as $D = \frac{1}{2}\sum_j \mu_j v_j^2$, where v_j denotes the tangential speed of a rolling bearing relative to its axis, or the sliding speed of a linear bearing. The mechanism contains two stationary-axis upper bearings on each belt, four lower bearings whose axes travel with the horizontal gantry, two x-direction linear bearings and two y-direction linear bearings. We assume that bearings of a given type have the same coefficient.

The two upper bearings on the left belt have tangential speeds of magnitude $|r_m\dot{\theta}_l|$, and the two on the right belt have $|r_m\dot{\theta}_r|$. For each lower bearing the tangential speed relative to its moving axis is $|\dot{x}|$, we assume that a purely vertical gantry translation does not make these bearings spin. Each linear bearing travels with speed $|\dot{x}|$ or $|\dot{y}|$ respectively. Consequently,

$$D = \mu_r r_m^2(\dot{\theta}_l^2 + \dot{\theta}_r^2) + (2\mu_r + \mu_{l,x})\dot{x}^2 + \mu_{l,y}\dot{y}^2. \tag{11}$$

Applying the Euler–Lagrange equations,

$$\frac{d}{dt}\frac{\partial K}{\partial \dot{\theta}_j} + \frac{\partial D}{\partial \dot{\theta}_j} = K_m i_{m,j}, \qquad j \in \{l, r\}, \tag{12}$$

we obtain

$$\begin{aligned} J_{\text{eff}}\ddot{\theta}_l + a\dot{\theta}_l + c\dot{\theta}_r &= K_m i_{m,l}, \\ J_{\text{eff}}\ddot{\theta}_r + c\dot{\theta}_l + a\dot{\theta}_r &= K_m i_{m,r}, \end{aligned} \tag{13}$$

where

$$a = r_m^2\left(3\mu_r + \frac{\mu_{l,x} + \mu_{l,y}}{2}\right), \qquad c = r_m^2\left(\mu_r + \frac{\mu_{l,x} - \mu_{l,y}}{2}\right). \tag{14}$$

State-space representation Let $\omega_l = \dot{\theta}_l$ and $\omega_r = \dot{\theta}_r$. We define

$$\mathbf{z} = \begin{bmatrix}\theta_l & \theta_r & \omega_l & \omega_r & i_{m,l} & i_{m,r}\end{bmatrix}^\top, \qquad \mathbf{u} = \begin{bmatrix}u_{m,l} & u_{m,r}\end{bmatrix}^\top. \tag{15}$$

Combining Eqs. (6) and (13), the requested state-space model is $\dot{\mathbf{z}} = A\mathbf{z} + B\mathbf{u}$, with

$$A = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & -a/J_{\text{eff}} & -c/J_{\text{eff}} & K_m/J_{\text{eff}} & 0 \\ 0 & 0 & -c/J_{\text{eff}} & -a/J_{\text{eff}} & 0 & K_m/J_{\text{eff}} \\ 0 & 0 & 0 & 0 & -R_m/L_m & 0 \\ 0 & 0 & 0 & 0 & 0 & -R_m/L_m \end{bmatrix},$$
$$B = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 1/L_m & 0 \\ 0 & 1/L_m \end{bmatrix}. \tag{16}$$

These equations define the continuous-time model used in the subsequent questions and in our Python implementation.

Q1.3 – Decoupling the x and y Dynamics

We introduce the following transformed currents and inputs:

$$\begin{aligned} i_x &= \frac{i_{m,l} + i_{m,r}}{2}, \quad u_x = \frac{u_{m,l} + u_{m,r}}{2}, \\ i_y &= \frac{i_{m,l} - i_{m,r}}{2}, \quad u_y = \frac{u_{m,l} - u_{m,r}}{2}. \end{aligned} \tag{17}$$

Horizontal dynamics Adding the two mechanical equations obtained in Q1.2 gives

$$J_{\text{eff}}(\ddot{\theta}_l + \ddot{\theta}_r) + (a+c)(\dot{\theta}_l + \dot{\theta}_r) = K_m(i_{m,l} + i_{m,r}). \tag{18}$$

Using the kinematic transformation and the definition of i_x, we obtain

$$J_{\text{eff}}\ddot{x} + (a+c)\dot{x} = r_m K_m i_x. \tag{19}$$

Similarly, adding the electrical equations and dividing by two yields

$$L_m \frac{d}{dt} i_x = u_x - R_m i_x. \tag{20}$$

Choosing the state vector $\mathbf{x}_x = [x, \dot{x}, i_x]^\top$, the horizontal state-space model becomes

$$\boxed{\dot{\mathbf{x}}_x = \begin{bmatrix} 0 & 1 & 0 \\ 0 & -\dfrac{a+c}{J_{\text{eff}}} & \dfrac{r_m K_m}{J_{\text{eff}}} \\ 0 & 0 & -\dfrac{R_m}{L_m} \end{bmatrix} \mathbf{x}_x + \begin{bmatrix} 0 \\ 0 \\ \dfrac{1}{L_m} \end{bmatrix} u_x.} \tag{21}$$

Vertical dynamics Subtracting the right motor equation from the left motor equation gives

$$J_{\text{eff}}(\ddot{\theta}_l - \ddot{\theta}_r) + (a-c)(\dot{\theta}_l - \dot{\theta}_r) = K_m(i_{m,l} - i_{m,r}). \tag{22}$$

Using the definitions of y and i_y, this becomes

$$J_{\text{eff}}\ddot{y} + (a-c)\dot{y} = r_m K_m i_y. \tag{23}$$

The corresponding electrical equation is

$$L_m \frac{d}{dt} i_y = u_y - R_m i_y. \tag{24}$$

With $\mathbf{x}_y = [y, \dot{y}, i_y]^\top$, the vertical state-space model is

$$\boxed{\dot{\mathbf{x}}_y = \begin{bmatrix} 0 & 1 & 0 \\ 0 & -\dfrac{a-c}{J_{\text{eff}}} & \dfrac{r_m K_m}{J_{\text{eff}}} \\ 0 & 0 & -\dfrac{R_m}{L_m} \end{bmatrix} \mathbf{x}_y + \begin{bmatrix} 0 \\ 0 \\ \dfrac{1}{L_m} \end{bmatrix} u_y.} \tag{25}$$

The damping coefficients, using the friction assumptions of Q1.2, simplify to

$$a+c=r_m^2(4\mu_r+\mu_{l,x}), \qquad a-c=r_m^2(2\mu_r+\mu_{l,y}). \tag{26}$$

The resulting state-space descriptions are independent, the horizontal dynamics depend only on $\mathbf{x}_x$ and u_x, while the vertical dynamics depend only on $\mathbf{x}_y$ and u_y. The original six-state system has therefore been transformed into two decoupled three-state systems.

Q1.4 – Drawing shapes

To demonstrate the CoreXY model, we prescribe two closed planar trajectories: an ellipse and a cardioid. We determine the required voltages by inverse dynamics and then simulate the decoupled state-space models of Q1.3.

Desired trajectories For the ellipse, we choose

$$x_d(t)=A\cos(\omega t), \qquad y_d(t)=B\sin(\omega t), \qquad \omega=\frac{2\pi}{T}, \tag{27}$$

where A and B are its semi-axes and T is the traversal period. For the second shape, we chose to draw a cardioid.[1] Its desired trajectory is

$$r(t)=R\big(1-\cos(\omega t)\big), \qquad x_d(t)=r(t)\cos(\omega t), \qquad y_d(t)=r(t)\sin(\omega t). \tag{28}$$

Input design by inverse dynamics From Q1.3, the dynamics are

$$\begin{aligned} J_{\text{eff}}\ddot{x}+b_x\dot{x}&=r_mK_mi_x, \quad L_m\frac{d}{dt}i_x=u_x-R_mi_x,\\ J_{\text{eff}}\ddot{y}+b_y\dot{y}&=r_mK_mi_y, \quad L_m\frac{d}{dt}i_y=u_y-R_mi_y, \end{aligned} \tag{29}$$

with $b_x=a+c$, and $b_y=a-c$.

For either coordinate $q\in\{x,y\}$, we define $b_q=b_x$ or b_y and the corresponding desired trajectory q_d. The required transformed current follows from the mechanical equation:

$$i_{q,d}(t)=\frac{J_{\text{eff}}\ddot{q}_d(t)+b_q\dot{q}_d(t)}{r_mK_m}. \tag{30}$$

Substituting this current and its derivative into the electrical equation gives the transformed voltage input

$$\boxed{u_q(t)=\frac{L_m\big(J_{\text{eff}}q_d^{(3)}+b_q\ddot{q}_d\big)+R_m\big(J_{\text{eff}}\ddot{q}_d+b_q\dot{q}_d\big)}{r_mK_m}}. \tag{31}$$

The voltages applied to the actual motors are obtained by inverting the transformation used in Q1.3:

$$\boxed{u_{m,l}(t)=u_x(t)+u_y(t), \qquad u_{m,r}(t)=u_x(t)-u_y(t).} \tag{32}$$

[1] E. W. Weisstein, "Cardioid," *Wolfram MathWorld* : `https://mathworld.wolfram.com/Cardioid.html`.

Simulation and results We integrate both three-state systems using Python's `solve_ivp` function and apply the motor voltages derived above. The initial position and velocity of each axis are chosen to coincide with the desired trajectory, the initial currents are set to $i_{x,d}(0)$ and $i_{y,d}(0)$.

The parameter values that we have selected for the simulation are summarized in Table 1 and were used to simulate both trajectories.

Table 1: Parameters used for the simulation

Mechanical	$r_m = 0.01\,\mathrm{m}$, $m = 0.06\,\mathrm{kg}$, $J_m = 10^{-5}\,\mathrm{kg\,m^2}$
Electrical	$K_m = 0.07\,\mathrm{N\,m/A}$, $L_m = 0.01\,\mathrm{H}$, $R_m = 10\,\Omega$
Friction	$(\mu_r, \mu_{l,x}, \mu_{l,y}) = (0.003, 0.002, 0.001)\,\mathrm{N\,s/m}$
Ellipse	$A = 0.04\,\mathrm{m}$, $B = 0.025\,\mathrm{m}$
Cardioid	$R = 0.035\,\mathrm{m}$
Both	$T = 8\,\mathrm{s}$

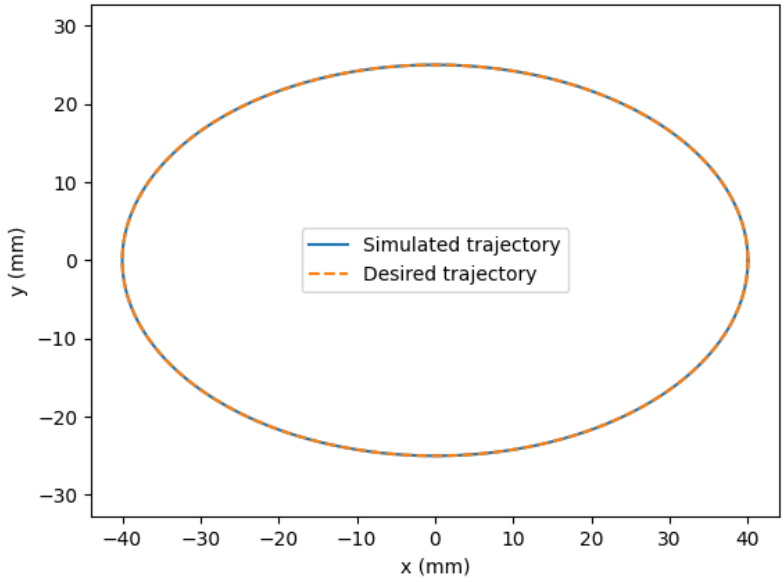


Figure 1: Ellipse: prescribed and simulated print-head trajectories.

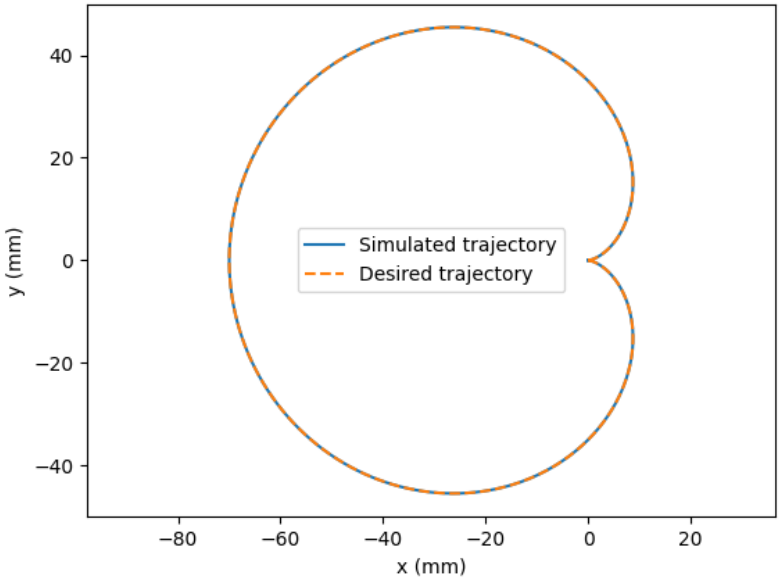


Figure 2: Cardioid: prescribed and simulated print-head trajectories.

Q1.5 – Model the Nozzle

We assume that the nozzle has a uniform temperature T_n and that all electrical power dissipated by the heating resistor is converted into heat. The thermal losses to the environment and the filament are combined into a single equivalent thermal resistance $R_{t,n}$, with reference temperature T_e.

Electrical model The electrical circuit consists of an inductor L_n connected in series with a parallel resistor-capacitor circuit. We denote by i_n the current through the inductor and by v_n the voltage across the resistor R_n and capacitor C_n.

Applying Kirchhoff's voltage law gives

$$u_n = L_n \frac{di_n}{dt} + v_n. \tag{33}$$

Therefore,

$$\frac{di_n}{dt} = \frac{u_n - v_n}{L_n}. \tag{34}$$

Applying Kirchhoff's current law at the node connecting the three components gives

$$i_n = C_n \frac{dv_n}{dt} + \frac{v_n}{R_n}. \tag{35}$$

Rearranging,

$$\frac{dv_n}{dt} = \frac{1}{C_n}\left(i_n - \frac{v_n}{R_n}\right). \tag{36}$$

Thermal model The electrical power converted into heat by the resistor is given by Joule's law

$$P_{\text{heat}} = \frac{v_n^2}{R_n}. \tag{37}$$

The heat loss to the surroundings is modelled using the equivalent thermal resistance

$$P_{\text{loss}} = \frac{T_n - T_e}{R_{t,n}}. \tag{38}$$

Applying the energy balance to the nozzle gives

$$C_{t,n}\frac{dT_n}{dt} = P_{\text{heat}} - P_{\text{loss}}. \tag{39}$$

Substituting the expressions for the heating power and the thermal losses,

$$\frac{dT_n}{dt} = \frac{1}{C_{t,n}}\left(\frac{v_n^2}{R_n} - \frac{T_n - T_e}{R_{t,n}}\right). \tag{40}$$

State-space representation We choose the state vector

$$\mathbf{x}_n = \begin{bmatrix} T_n & i_n & v_n \end{bmatrix}^\top, \tag{41}$$

with the applied voltage u_n as the system input.
Combining the electrical and thermal equations, the complete state-space model becomes

$$\boxed{\dot{\mathbf{x}}_n = \begin{bmatrix} \dfrac{1}{C_{t,n}}\left(\dfrac{v_n^2}{R_n} - \dfrac{T_n - T_e}{R_{t,n}}\right) \\ \dfrac{u_n - v_n}{L_n} \\ \dfrac{1}{C_n}\left(i_n - \dfrac{v_n}{R_n}\right) \end{bmatrix}}. \tag{42}$$

Q1.6 – Cooling of the Printed Filament

We model the first printed layer as a one-dimensional filament deposited along a closed elliptic path. The spatial coordinate s denotes arc length along this path, and $T(s,t)$ denotes the local filament temperature. We assume uniform temperature across each cross-section, constant material properties, a square cross-section of side w, and constant temperatures of the nozzle, print bed and surrounding air. Heat conduction takes place along the filament, while heat is exchanged with the air and the heated print bed.

Geometry and growing domain Let L be the total perimeter of the ellipse. For a constant deposition speed v, the printed length and printing time are

$$\ell(t) = \min(vt, L), \qquad t_p = \frac{L}{v}, \qquad \Omega(t) = [0, \ell(t)]. \tag{43}$$

Energy balance and PDE For a segment of length ds, the volume is $w^2 ds$. Three faces of the square cross-section are exposed to air at temperature T_e, while its lower face is in contact with the heated bed at temperature T_b. Introducing heat-transfer coefficients h_a and h_b, the losses (positive for heat leaving the filament) are

$$dP_a = 3h_a w(T - T_e)\,ds, \qquad dP_b = h_b w(T - T_b)\,ds. \tag{44}$$

Fourier's law gives the axial heat flow $q_s = -\kappa w^2 \partial T/\partial s$, where κ is the filament thermal conductivity. Conservation of energy gives

$$\rho c_p w^2 \frac{\partial T}{\partial t} = \kappa w^2 \frac{\partial^2 T}{\partial s^2} - 3h_a w(T - T_e) - h_b w(T - T_b), \tag{45}$$

where ρ is the density and c_p is the specific heat capacity. Dividing by $\rho c_p w^2$ gives us the one-dimensional heat equation

$$\boxed{\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial s^2} - \beta(T - T_{\text{eq}}),} \tag{46}$$

with

$$\alpha = \frac{\kappa}{\rho c_p}, \qquad \beta = \frac{3h_a + h_b}{\rho c_p w}, \qquad T_{\text{eq}} = \frac{3h_a T_e + h_b T_b}{3h_a + h_b}. \tag{47}$$

Deposition and boundary conditions During printing, we impose no axial heat flux at the original free end, and assign the freshly deposited material the temperature T_n at the moving end. Once the nozzle completes the closed loop, the end points are physically connected, giving periodic boundary conditions:

$$T(0,t) = T(L,t), \qquad \frac{\partial T}{\partial s}(0,t) = \frac{\partial T}{\partial s}(L,t), \qquad t \geq t_p. \tag{48}$$

Numerical method The full path is divided into N cells of length $\Delta s = L/N$. For each deposited cell, the heat equation is integrated using explicit finite differences:

$$\boxed{T_i^{k+1} = T_i^k + \Delta t \left[\alpha \frac{T_{i+1}^k - 2T_i^k + T_{i-1}^k}{\Delta s^2} - \beta(T_i^k - T_{\text{eq}}) \right].} \tag{49}$$

During printing, a new cell is activated at T_n as the nozzle progresses. No conduction is allowed into the unprinted region, which is equivalent to zero axial heat flux beyond the current end cell between depositions. After t_p, periodic neighbours are used to represent the closed loop.

Simulation and results The material and heat-transfer parameter values that we have selected for the simulation are summarized in. The ellipse dimensions and deposition duration are consistent with the trajectory used in Q1.4.

Table 2: Parameters used for the simulation

Ellipse semi-axes	$a_e = 0.04\,\text{m}$, $b_e = 0.025\,\text{m}$
Filament side	$w = 8 \times 10^{-4}\,\text{m}$
Density, heat capacity	$\rho = 1250\,\text{kg/m}^3$, $c_p = 1800\,\text{J/(kg K)}$
Thermal conductivity	$\kappa = 0.2\,\text{W/(m K)}$
Air, bed exchange	$h_a = 10$, $h_b = 60\,\text{W/(m}^2\,\text{K)}$
Nozzle, air, bed temperatures	$T_n = 465\,\text{K}$, $T_e = 300\,\text{K}$, $T_b = 333.15\,\text{K}$
Print duration	$t_p = 8\,\text{s}$
Discretization	$N = 160$, $\Delta t = 0.05\,\text{s}$

Our criteria for the requirement that the printed layer becomes nearly uniform is the following, we stop the simulation when the complete loop satisfies

$$\max_i T_i - \min_i T_i \leq 1K. \tag{50}$$

Figure 3 shows how the temperature domain grows during the first 8 s and then cools along the completed loop. Figure 4 shows the minimum and maximum temperatures. The temperature spread becomes smaller than 1 K at $t \approx 82.80$ s after printing starts, or 74.80 s after the first layer is completed.

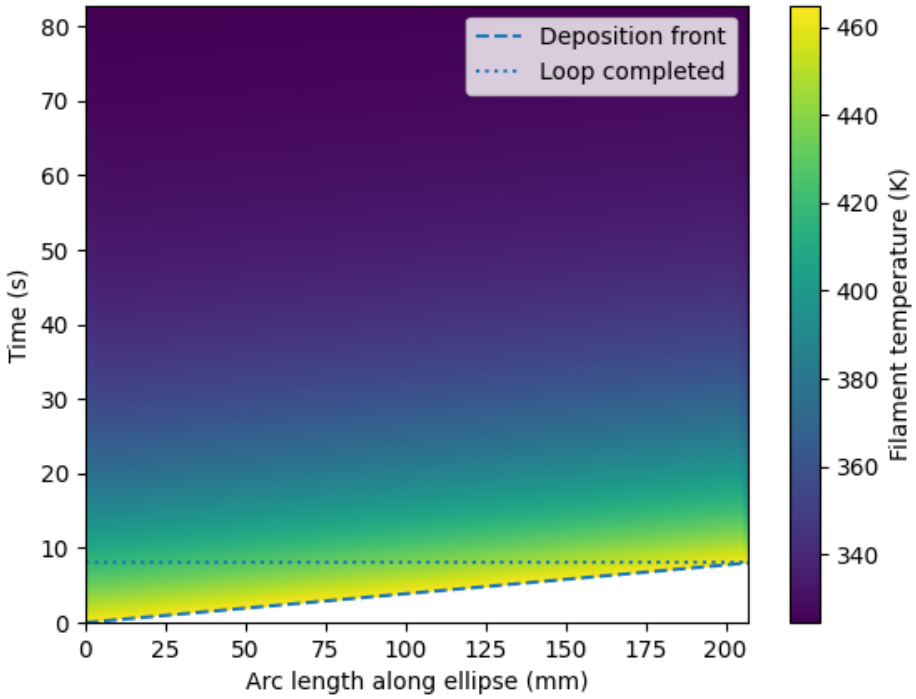


Figure 3: Temperature along the filament during progressive deposition and early cooling.

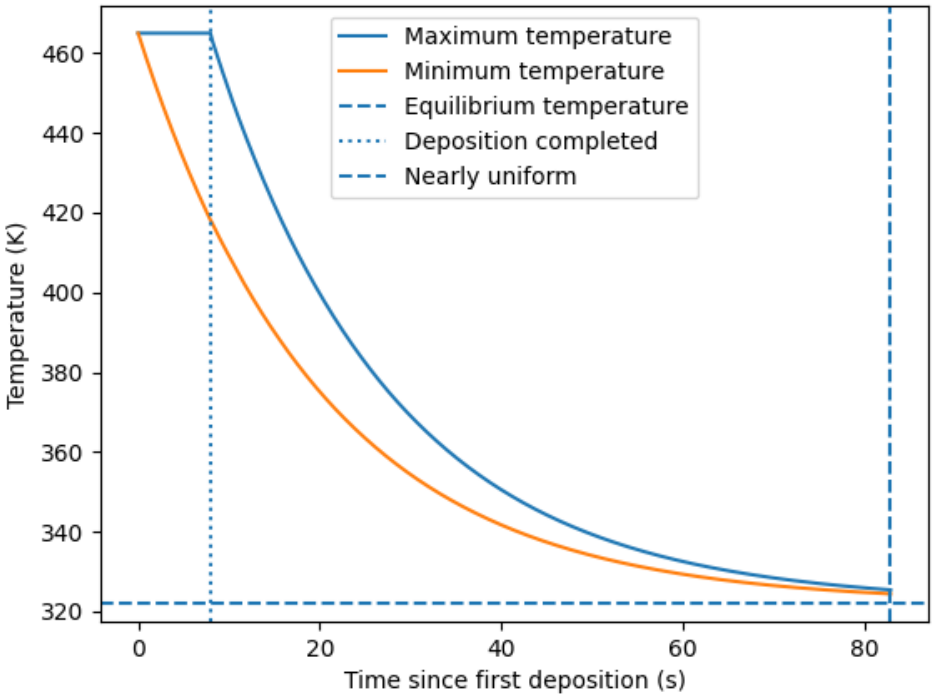


Figure 4: Evolution of the minimum and maximum temperatures, showing deposition completion and the time at which the temperature becomes nearly uniform.

Q2.1 – Finite State Machine for Filament Replacement

The filament replacement procedure is modelled using a Finite State Machine. Two independent discrete events can trigger the process. C representing a colour change request, and F indicating that the filament spool is empty.

Since both events require the same replacement procedure, the transition from the printing state is triggered by $C \vee F$. At this transition, printing is stopped and the current print-head position (x_s, y_s) is saved.

Ours FSM consists of seven states, shown in Figure 5.

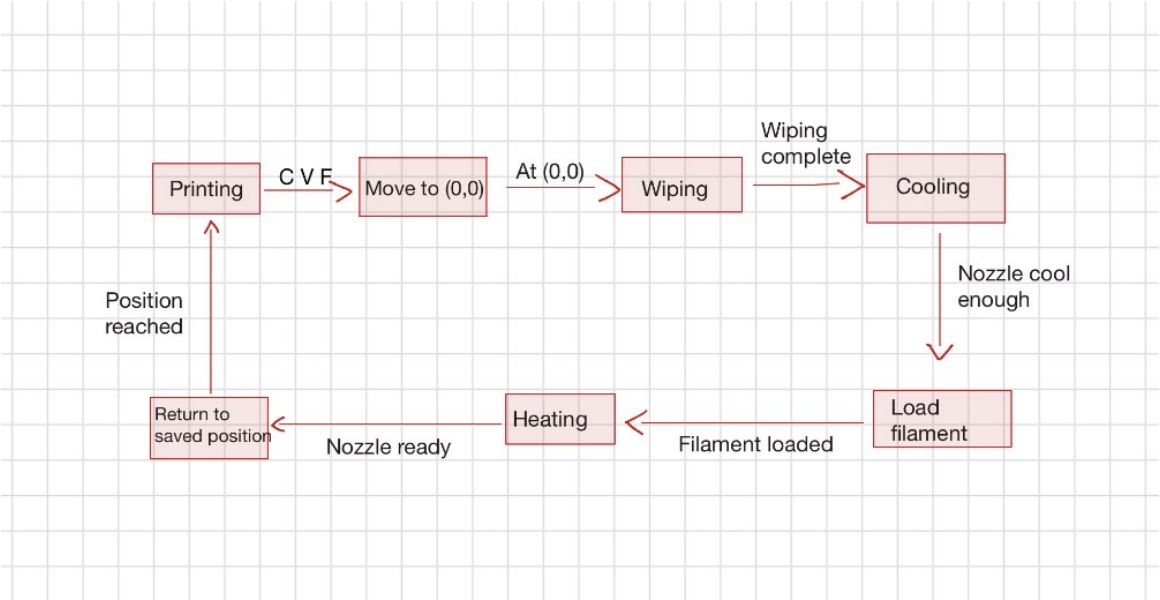


Figure 5: Finite State Machin for the filament procedure.

Q2.2 – Automata: Parked Detection

The print head is parked at the origin when it has reached both the left boundary ($x = 0$) and the bottom boundary ($y = 0$). As no end-of-line sensors are available, these boundaries are detected using the commanded motion and the motor angular-velocity measurements. The input $(H,0)$ indicates that a horizontal command towards the left boundary produces no motion, whereas $(V,0)$ indicates the same for a vertical command towards the bottom boundary. The symbol $+$ denotes detected motion. We assume that the corresponding voltage command is applied and that the absence of motion indicates contact with the respective boundary.

We define a deterministic finite automaton

$$\mathscr{M} = (Q, \Sigma, \delta, Q_0, F), \tag{51}$$

where

$$\begin{aligned} Q &= \{q_0, q_H, q_V, q_P\}, \\ \Sigma &= \{(H,+),(H,0),(V,+),(V,0)\}, \\ Q_0 &= \{q_0\}, \qquad F = \{q_P\}. \end{aligned} \tag{52}$$

The initial state q_0 means that neither boundary has been detected. State q_H indicates that the left boundary has been reached, while q_V indicates that the bottom boundary has been reached. The accepting state q_P represents the head being parked at $(0,0)$.

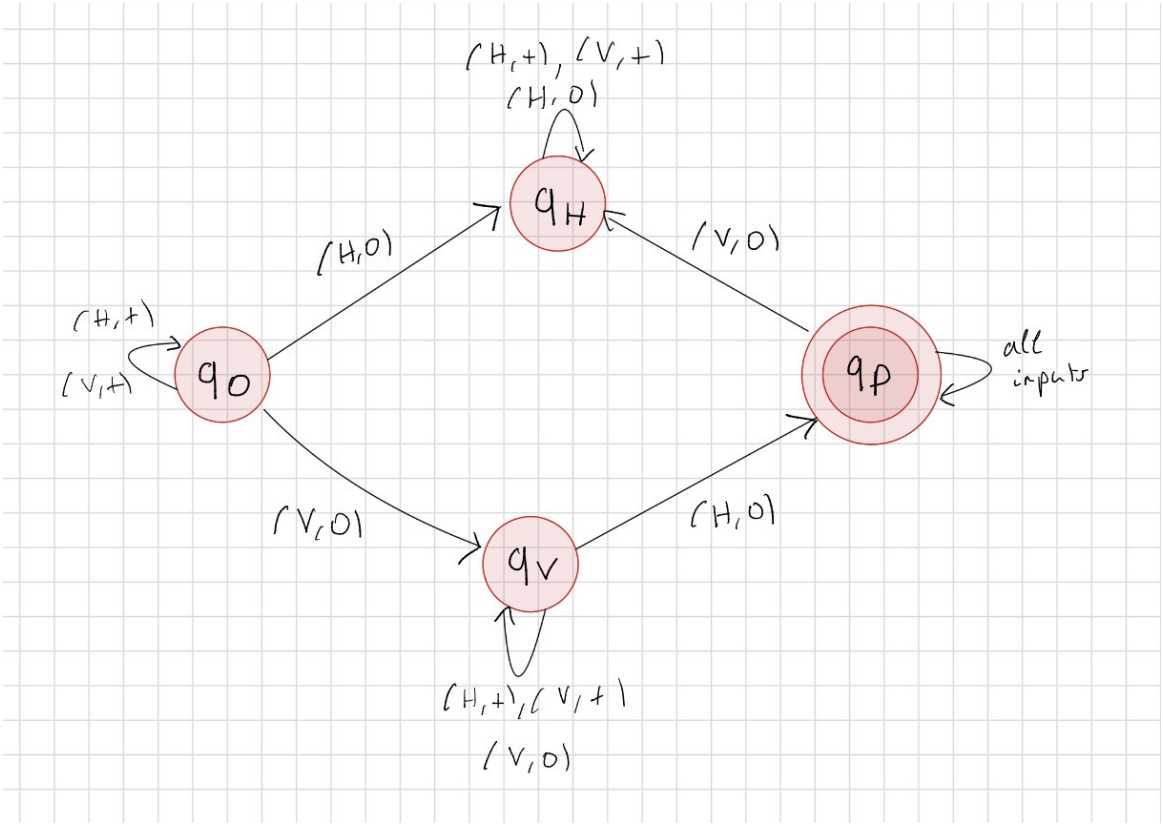


Figure 6: Deterministic finite automaton for detecting the parked print head.

The automaton accepts a sequence once it has observed both $(H,0)$ and $(V,0)$, in either order. In particular, the example sequence from the project statement,

$$\sigma = (H,+)(V,+)(H,+)(H,0)(V,+)(V,0), \tag{53}$$

produces the state sequence

$$q_0 \to q_0 \to q_0 \to q_0 \to q_H \to q_H \to q_P. \tag{54}$$

Q3.1 – Filament Replacement

Hybrid system model We combine the motor dynamics from Q1.2, the nozzle model from Q1.5 and the filament-replacement FSM from Q2.1 into a hybrid system. We define

$$H = (Q, X, f, \text{Init}, \text{Inv}, E, G, R), \tag{55}$$

where Q is the set of discrete modes, X the continuous state space, f the continuous dynamics, Init the initial states, Inv the invariants, E the transitions, G the guards and R the reset relations.

The physical state and input vectors are

$$\mathbf{z} = \begin{bmatrix} \theta_l & \theta_r & \omega_l & \omega_r & i_{m,l} & i_{m,r} & T_n & i_n & v_n \end{bmatrix}^\top, \qquad \mathbf{u} = \begin{bmatrix} u_{m,l} & u_{m,r} & u_n \end{bmatrix}^\top. \tag{56}$$

The nine equations from Q1.2 and Q1.5 are combined into $\dot{\mathbf{z}} = f(\mathbf{z}, \mathbf{u})$. The simulation also stores the position at which printing is interrupted and the starting time of each operation.

Discrete modes The replacement procedure is represented by

$$Q = \{\text{Printing}, \text{Move}_V, \text{Move}_H, \text{Wiping}, \text{Cooling}, \text{Loading}, \text{Heating}, \text{Return}\}.$$

The initial mode is Printing. When a colour-change request C or an empty-spool event F occurs, printing stops and the current position (x_s, y_s) is saved. The head first moves vertically to $(x_s, 0)$ and then horizontally to the wiping location $(0,0)$. After wiping, the nozzle heater is switched off, the new filament is loaded and the nozzle is reheated. Finally, the head returns to (x_s, y_s) and printing resumes. The two movement modes distinguish the vertical and horizontal parking phases.

Motor and nozzle inputs To simulate these movements, we must determine the motor voltages rather than directly impose the print-head position. We therefore reuse the inverse dynamics of Q1.4, which gives the required voltages once a desired trajectory is specified.

For either Cartesian coordinate $\xi \in \{x, y\}$, consider a movement from ξ_a to ξ_b in a duration D. We introduce the normalized time

$$s = \frac{t - t_0}{D}, \qquad 0 \le s \le 1, \tag{57}$$

where t_0 is the starting time, and write the trajectory as

$$\xi_d(t) = \xi_a + (\xi_b - \xi_a) p(s). \tag{58}$$

To start and finish each movement at rest, with zero acceleration and hence zero desired motor current, we impose

$$\begin{aligned} p(0) &= 0, \quad p(1) = 1, \\ p'(0) &= 0, \quad p'(1) = 0, \\ p''(0) &= 0, \quad p''(1) = 0. \end{aligned} \tag{59}$$

We choose a fifth-degree interpolation polynomial[2], which has six coefficients to satisfy these six conditions:

$$p(s) = a_0 + a_1 s + a_2 s^2 + a_3 s^3 + a_4 s^4 + a_5 s^5. \tag{60}$$

The conditions at $s = 0$ give $a_0 = a_1 = a_2 = 0$, while those at $s = 1$ give

$$\begin{cases} a_3 + a_4 + a_5 = 1, \\ 3a_3 + 4a_4 + 5a_5 = 0, \\ 6a_3 + 12a_4 + 20a_5 = 0. \end{cases} \tag{61}$$

Solving yields $a_3 = 10$, $a_4 = -15$ and $a_5 = 6$. Thus,

$$\boxed{\xi_d(t) = \xi_a + (\xi_b - \xi_a)(10s^3 - 15s^4 + 6s^5).} \tag{62}$$

This choice makes it possible to stop the head between movements. Its velocity, acceleration and third derivative are used in the inverse dynamics:

$$\begin{aligned} i_{\xi,d} &= \frac{J_{\text{eff}} \ddot{\xi}_d + b_\xi \dot{\xi}_d}{r_m K_m}, \qquad u_\xi = L_m \frac{d}{dt} i_{\xi,d} + R_m i_{\xi,d}, \\ u_{m,l} &= u_x + u_y, \qquad\qquad u_{m,r} = u_x - u_y, \end{aligned} \tag{63}$$

For the simulation, we chose that ach parking and return movement lasts $D = 1\,\text{s}$, the initial printing movement covers 12 mm in 1.5 s. Outside planned movements, the motor voltages are zero.

[2]The use of a polynomial trajectory interpolation was suggested with the assistance of an AI tool

For the nozzle, $u_n = 0$ during Cooling and Loading, and $u_n = u_{\text{heat}}$ during Heating. In the remaining modes, a constant holding voltage maintains the nominal printing temperature. From the nozzle equations of Q1.5, the steady-state conditions $\dot{T}_n = \frac{d}{dt} i_n = \dot{v}_n = 0$ give $v_n = u_n$ and

$$\frac{u_n^2}{R_n} = \frac{T_{\text{ready}} - T_e}{R_{t,n}}. \tag{64}$$

Taking the positive root gives

$$\boxed{u_{\text{hold}} = \sqrt{\frac{R_n(T_{\text{ready}} - T_e)}{R_{t,n}}}.} \tag{65}$$

Guards, invariants and reset maps The discrete modes change when the corresponding guards are satisfied. We use the elapsed time $\tau = t - t_0$, with $\dot{\tau} = 1$ and $\tau^+ = 0$ at each new operation. Position and speed tolerances ε_p and ε_v define the motion guards:

$$\begin{aligned} G_V &: \ \tau \geq D \ \wedge \ |y| \leq \varepsilon_p \ \wedge \ |\dot{y}| \leq \varepsilon_v, \\ G_H &: \ \tau \geq D \ \wedge \ |x| \leq \varepsilon_p \ \wedge \ |\dot{x}| \leq \varepsilon_v, \\ G_R &: \ \tau \geq D \ \wedge \ \sqrt{(x-x_s)^2 + (y-y_s)^2} \leq \varepsilon_p \ \wedge \ \sqrt{\dot{x}^2 + \dot{y}^2} \leq \varepsilon_v. \end{aligned} \tag{66}$$

Here the positions and velocities are reconstructed from the motor states using Q1.1. The simulation represents arrival at the boundaries through planned movements, rather than modelling the physical motor stalls considered in Q2.2.

Table 3 gives the transitions, guards, invariants and resets. The physical states remain continuous, $\mathbf{z}^+ = \mathbf{z}^-$, while the saved position, time reference and movement targets are updated when required. The nozzle voltage is determined by the active mode, not by resetting its physical state.

Table 3: Mode transitions, invariants and reset maps for filament replacement

Mode / next mode	Guard	Mode invariant	Reset at transition
Printing → Move_V	$C \vee F$	X	Save $(x_s, y_s) = (x, y)$; $\tau^+ = 0$
Move_V → Move_H	G_V	X	Set horizontal target; $\tau^+ = 0$
Move_H → Wiping	G_H	X	$\tau^+ = 0$
Wiping → Cooling	$\tau \geq t_w$	$\tau \leq t_w$	$\tau^+ = 0$
Cooling → Loading	$T_n \leq T_{\text{safe}}$	$T_n \geq T_{\text{safe}}$	$\tau^+ = 0$
Loading → Heating	$\tau \geq t_L$	$\tau \leq t_L$	$\tau^+ = 0$
Heating → Return	$T_n \geq T_{\text{ready}}$	$T_n \leq T_{\text{ready}}$	Set saved-position target; $\tau^+ = 0$
Return → Printing	G_R	X	Set printing target; $\tau^+ = 0$

Numerical simulation and results We use the same mechanical and electrical parameters as in Q1.4. The additional parameter values selected for the nozzle and the filament replacement simulation are summarized in Table 4.

Table 4: Parameters used for the simulation

Parameter	Symbol	Value
Nozzle inductance	L_n	$0.010\,\mathrm{H}$
Nozzle capacitance	C_n	$0.002\,\mathrm{F}$
Electrical resistance	R_n	$8\,\Omega$
Thermal capacitance	$C_{t,n}$	$0.6\,\mathrm{J/K}$
Thermal resistance	$R_{t,n}$	$9\,\mathrm{K/W}$
Ambient temperature	T_e	$300\,\mathrm{K}$
Safe temperature	T_{safe}	$335\,\mathrm{K}$
Printing temperature	T_{ready}	$465\,\mathrm{K}$
Heating voltage	u_{heat}	$18\,\mathrm{V}$
Position tolerance	ε_p	$0.5\,\mathrm{mm}$
Velocity tolerance	ε_v	$0.003\,\mathrm{m/s}$
Wiping duration	t_w	$0.7\,\mathrm{s}$
Loading duration	t_L	$0.7\,\mathrm{s}$
Replacement request time	t_{request}	$1.5\,\mathrm{s}$

A replacement request at t_{request}, representing either event C or F, initiates the filament replacement procedure.

The continuous dynamics are implemented in `sys(t,z)` and integrated with Python's `solve_ivp`, following the approach presented in Chapter 6. For each mode, an event function describes the active guard. For example, $T_n - T_{\text{safe}} = 0$ triggers the transition from Cooling to Loading. The solver stops when an event is detected, the discrete mode and auxiliary variables are updated, and integration resumes with the new inputs.

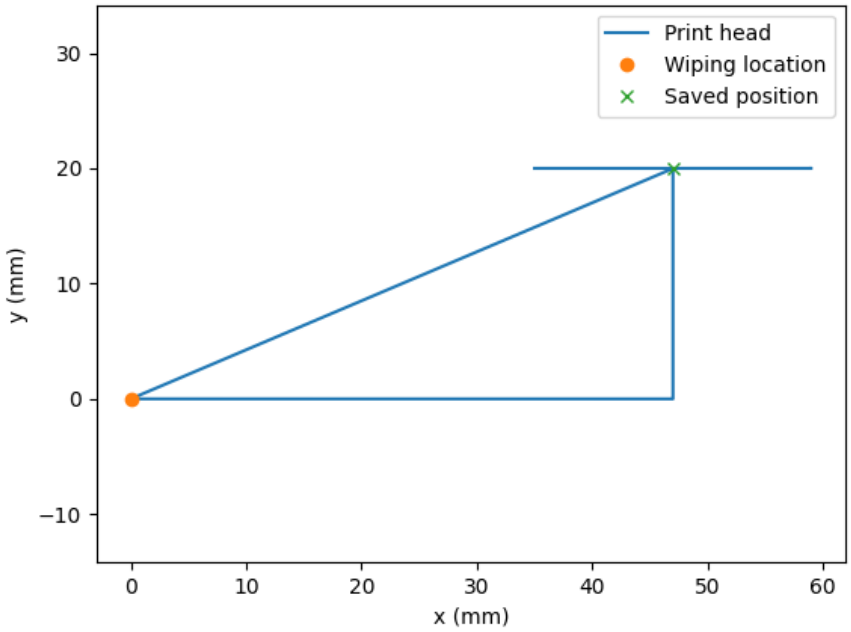


Figure 7: Print-head motion during filament replacement, showing parking at the origin and return to the saved position.

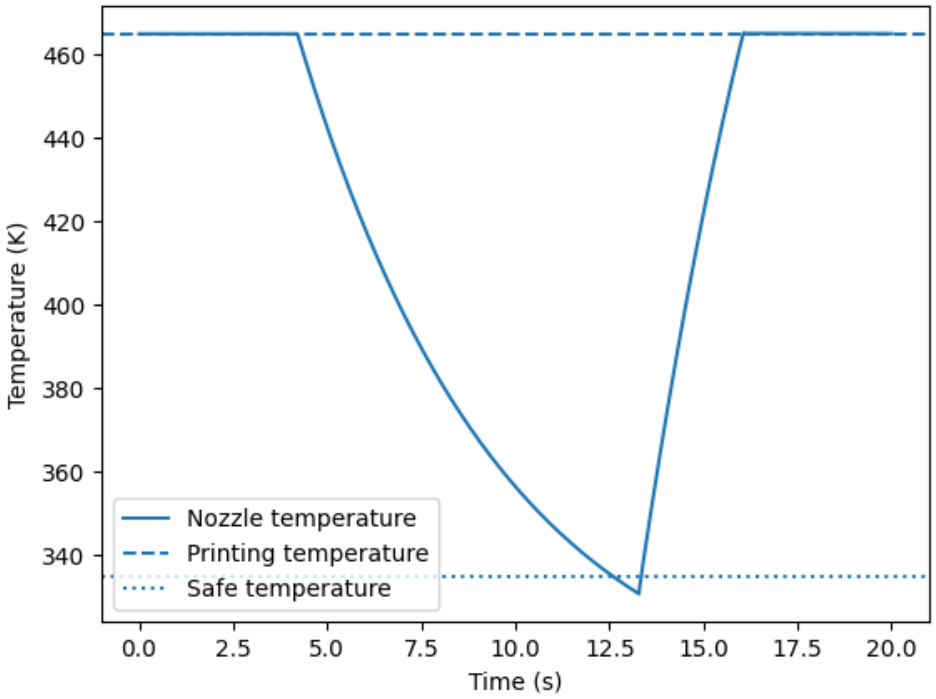


Figure 8: Nozzle temperature during cooling, loading and reheating.

The head reaches the origin at 3.50 s and wiping ends at 4.20 s. The nozzle reaches the safe loading temperature at 12.58 s and the printing temperature at 16.05 s. Printing resumes at approximately 17.05 s.

Q3.2 – Draw a Square

Hybrid system extension We extend the hybrid model of Q3.1 with four successive drawing modes. The physical dynamics remain unchanged and the nozzle is maintained at its printing temperature using u_{hold}. Each mode prescribes one side of the square, and transitions are determined from the motor-angle measurements.

Square trajectory and motor angles Let (x_0, y_0) denote the initial print-head position and d the side length. The target corners are

$$\begin{aligned} P_1 &= (x_0 + d, y_0), \quad P_2 = (x_0 + d, y_0 + d), \\ P_3 &= (x_0, y_0 + d), \quad P_4 = (x_0, y_0). \end{aligned} \tag{67}$$

Using the kinematics from Q1.1, the motor angles corresponding to a target (x_d, y_d) are

$$\theta_{l,d} = \frac{x_d + y_d}{r_m}, \qquad \theta_{r,d} = \frac{x_d - y_d}{r_m}. \tag{68}$$

The resulting targets, measured relative to the initial motor angles, are given in Table 5. We assume a known initial angular reference, no position sensors are used to detect the corners.

Table 5: Motor-angle targets, measured relative to the initial angles

Mode	Drawing motion	$\theta_{l,d} - \theta_l(0)$	$\theta_{r,d} - \theta_r(0)$
q_1	Right	d/r_m	d/r_m
q_2	Up	$2d/r_m$	0
q_3	Left	d/r_m	$-d/r_m$
q_4	Down	0	0

Motor inputs and hybrid transitions As in Q3.1, each side is described by the fifth-degree polynomial trajectory, with a duration $D = 1.5\,\text{s}$ and endpoints given by consecutive corners. The motor voltages are calculated using the inverse dynamics of Q1.4. The discrete modes are

$$Q = \{q_1, q_2, q_3, q_4, q_{\text{done}}\}.$$

The elapsed time since the start of a side is $\tau = t - t_i$, with $\dot{\tau} = 1$. A transition from q_i is enabled when the duration has elapsed and both measured motor angles are sufficiently close to their targets:

$$\boxed{G_i: \quad \tau \geq D \ \wedge \ \max\big(|\theta_l - \theta_{l,d}^{(i)}|, |\theta_r - \theta_{r,d}^{(i)}|\big) < \varepsilon_\theta.} \tag{69}$$

We take $\text{Inv}(q_i) = X$. When G_i is satisfied, the next side is selected, or the mode becomes q_{done} after the fourth side. The physical state is unchanged at the transition:

$$\mathbf{z}^+ = \mathbf{z}^-, \qquad q^+ = q_{\text{next}}, \qquad \tau^+ = 0. \tag{70}$$

Numerical simulation and results We reuse the physical parameters of Q3.1. The additional parameters selected for the Q3.2 simulation are summarized in Table 6.

As in Q3.1, the differential equations are integrated with `solve_ivp`. A terminal event detects when the elapsed-time and motor-angle conditions in G_i are satisfied,the solver then restarts in the next drawing mode. The positions shown in the plots are reconstructed from the motor angles using Q1.1.

Table 6: Parameters used for the simulation

Parameter	Symbol	Value
Initial position	(x_0, y_0)	$(0.04, 0.04)\,\mathrm{m}$
Square side length	d	$0.04\,\mathrm{m}$
Movement duration	D	$1.5\,\mathrm{s}$
Angular tolerance	ε_θ	$5 \times 10^{-4}\,\mathrm{rad}$

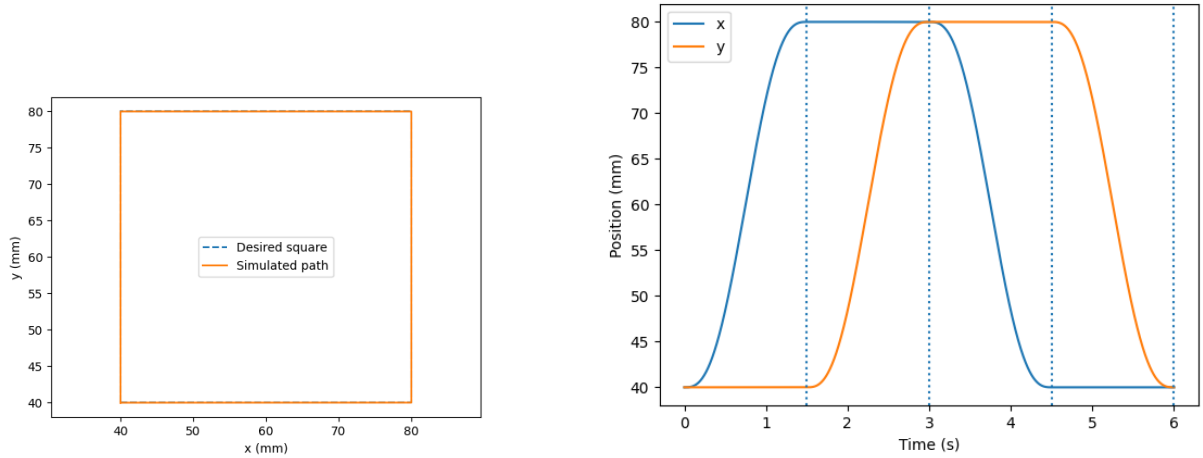


Figure 9: Left: desired square and simulated print-head trajectory. Right: positions reconstructed from motor angles, with switching instants marked by dotted lines.

The four corner detections occur at approximately 1.50, 3.00, 4.50 and 6.00 s. Figure 9 compares the prescribed and simulated squares (left) and shows the reconstructed coordinates with the switching instants (right).